

Voici la version finale et formalisée du **Cahier des Charges (V2)** intégrant le nom de code **KeycePass**, l'arborescence multi-modules propre à l'architecture MVVM, la structure de la base de données, ainsi que l'intégralité de vos exigences fonctionnelles, non fonctionnelles et histoires utilisateur issues de la V1.

Ce document est structuré de manière académique et industrielle, prêt pour une présentation ou une évaluation.

CAHIER DES CHARGES : CONTRÔLE DE GESTION DE LA FICHE DE PRÉSENCE DES ÉTUDIANTS

Code UE : B2COM0602 — Projet Tutoré N°2 & Ingénierie des Exigences

Niveau : Bachelor 2 (B2) IT — Collège de Paris Supérieur / Keyce Informatique

Nom de l'Application : KeycePass

Date de spécification : Juin 2026

1. CONSTAT ACTUEL ET BESOINS UTILISATEURS

1.1. Constat actuel et problématique

Le système actuel de suivi des présences au sein de l'université repose exclusivement sur un support physique traditionnel (fiche d'émargement papier nominative). Ce mode de fonctionnement manuel engendre des failles et des vulnérabilités critiques affectant directement l'intégrité et la fiabilité des données de contrôle :

- **Fraude par complaisance (Tricherie) :** L'absence de mécanisme d'authentification à la signature permet à des étudiants présents de signer en lieu et place d'élèves absents, ou de s'en aller immédiatement après avoir signé.
- **Altération et perte de données :** Risque élevé de détérioration physique, de perte de fiches lors des transferts ou de falsification a posteriori.
- **Inflexibilité administrative :** Charge de travail chronophage pour le service du contrôle de gestion (saisie manuelle des absences), induisant des délais importants pour le traitement des quotas d'absences requis pour la validation des examens.

1.2. Besoins utilisateurs par typologie d'acteurs

- **L'Administration :** Centraliser le contrôle, sécuriser la liaison entre l'identité de l'étudiant et son terminal physique, et obtenir des rapports fiables pour l'application automatique du règlement intérieur.

- **L'Enseignant** : Disposer d'un moyen simple, rapide et infalsifiable pour attester la tenue effective de sa séance de cours sans empiéter sur le temps pédagogique.
- **Le Délégué de classe** : Disposer d'un outil d'agrégation instantané pour générer, valider et transmettre le rapport de fin de session à la scolarité.
- **L'Étudiant** : Pouvoir justifier de sa présence tout au long du cours de manière transparente, autonome et instantanée via son smartphone personnel.

2. EXIGENCES FONCTIONNELLES (EF)

Chaque spécification fonctionnelle est unique, traçable, mesurable et orientée "système", conformément aux normes de l'ingénierie logicielle.

ID	Catégorie	Description de l'Exigence Fonctionnelle
EF_01	Sécurité / Enrôlement	Le système doit capturer l'identifiant unique du terminal (UUID/Hardware ID) lors du premier scan et bloquer toute tentative de modification sans validation physique préalable de l'administration.
EF_02	Gestion des QR Codes	L'interface administrative doit permettre la génération d'un QR Code statique unique indexé par classe (filière/niveau) et par semestre.
EF_03	Traitement du Scan	L'application mobile doit décoder le QR code de la classe et comparer l'horodatage système local (sécurisé en HTTPS) lors du scan de début et du scan de fin de cours pour attribuer le statut final de l'étudiant.
EF_04	Règles des Statuts	Le système doit croiser obligatoirement les données des deux scans selon les règles suivantes : • Présent : Premier scan entre [0 - 15 min] ET second scan validé. • En retard : Premier scan après [> 15 min] ET second scan validé.

ID	Catégorie	Description de l'Exigence Fonctionnelle
		• Absent : Premier scan OU second scan manquant.
EF_05	Clôture Enseignant	L'application doit exiger le scan d'un code de validation par l'enseignant en fin de séance pour valider juridiquement la tenue de la séance et déclencher l'autorisation du scan de fin pour les étudiants.
EF_06	Rapport Délégué	L'interface du délégué doit compiler les données locales de la séance et exporter un rapport de présence structuré vers l'administration.

3. EXIGENCES NON FONCTIONNELLES (ENF)

ID	Critère Qualitatif	Description de l'Exigence Non Fonctionnelle
ENF_01	Robustesse Anti-Fraude	Le système doit invalider le scan de présence si le jeton d'authentification ou l'identifiant matériel de l'appareil ne correspond pas au registre unique lié à l'étudiant en base de données (<i>Device Binding</i>).
ENF_02	Performance / UI	Le décodage du QR Code via l'appareil photo (intégration CameraX) et l'affichage visuel du statut de l'étudiant doivent s'exécuter en moins de 1,5 seconde.
ENF_03	Sécurité des Données	Les identifiants uniques et jetons de session stockés sur le smartphone doivent être chiffrés localement via l'API <i>EncryptedSharedPreferences</i> d'Android.
ENF_04	Portabilité / Cible	L'application mobile doit être développée de manière native sous Android avec le framework Jetpack Compose (Material Design 3), compatible à partir d'Android 7.0 (API minimale 24).

4. CYCLE DE VIE DE LA SÉANCE ET DES DONNÉES

Le déroulement nominal d'une séance de cours et le traitement des données de présence suivent un cycle strict découpé en 5 phases :

1. **Phase d'Initialisation (Début de Semestre) :** L'administration génère le QR Code statique de la classe. L'étudiant installe l'application. Lors du premier scan, l'application extrait l'UUID matériel. L'administration valide physiquement et enregistre le couple [Matricule Étudiant <=> ID Téléphone] dans la base de données centrale.
2. **Phase du Premier Scan (Début de Cours) :** Le QR Code est mis à disposition dans la salle. Les étudiants effectuent leur premier scan. L'application enregistre l'heure : si le scan a lieu entre 0 et 15 minutes après le début officiel, l'étudiant est éligible au statut *Présent*. S'il scanne après 15 minutes, il est marqué comme potentiellement *En retard*.
3. **Phase de Consolidation (Pendant le Cours) :** Les données du premier pointage sont conservées en attente dans la base de données locale (Room Database) de l'application mobile. Un étudiant n'ayant pas fait ce premier scan est temporairement considéré comme absent.
4. **Phase du Deuxième Scan et Clôture (Fin de Cours) :** En fin de cours, l'enseignant effectue son scan de validation pour verrouiller la séance. Ce geste débloque simultanément le scan de fin de séance pour les étudiants. Chaque étudiant doit flasher à nouveau le code pour confirmer qu'il est resté jusqu'au bout. Le système calcule alors le statut final (*Présent*, *En retard* ou *Absent*).
5. **Phase d'Exportation et Archivage :** Le délégué de classe valide et envoie le rapport consolidé des doubles scans à l'administration via l'infrastructure réseau. Le logiciel de l'administration intègre automatiquement les données pour alimenter le tableau de bord du contrôle de gestion des absences.

5. USER STORIES (HISTOIRES UTILISATEUR)

- **US_01 : Enrôlement Initial du Téléphone (Anti-Fraude)**
 - *En tant qu' Étudiant,*
 - *je veux* que mon application transmette l'identifiant unique de mon téléphone à l'administration lors de mon tout premier scan,
 - *afin de* lier de manière exclusive mon appareil à mon identité pour le reste du semestre.
 - *Critères d'acceptation :* L'application bloque tout changement d'appareil. L'administration dispose d'un bouton de réinitialisation sécurisé en base de données en cas de force majeure.
- **US_02 : Premier Scan (Arrivée en Cours)**
 - *En tant qu' Étudiant,*
 - *je veux* scanner le QR code de ma classe dès mon arrivée au début du cours,
 - *afin d' enregistrer* mon heure d'arrivée dans le système.
 - *Critères d'acceptation :* Si l'heure du scan ≤ 15 min du début du cours, l'application pré-enregistre l'état "À l'heure". Si l'heure est > 15 min, l'application pré-enregistre l'état "En retard".
- **US_03 : Deuxième Scan (Validation de Sortie et Statut Final)**
 - *En tant qu' Étudiant,*
 - *je veux* scanner le QR code une seconde fois à la fin de la séance,
 - *afin de* valider définitivement ma présence au cours.
 - *Critères d'acceptation :* Si le premier scan était "À l'heure" ET que le scan de fin est validé $\rightarrow$ statut final = Présent. Si le premier scan était "En retard" ET que le scan de fin est validé $\rightarrow$ statut final = En retard. Si le scan de début OU le scan de fin est manquant $\rightarrow$ statut final = Absent.
- **US_04 : Clôture de Session Enseignant**
 - *En tant qu' Enseignant,*
 - *je veux* scanner le code de confirmation en fin de cours,

- *afin d' officialiser la tenue de la séance, de figer la liste et de permettre aux étudiants de faire leur second scan.*
- *Critères d'acceptation* : Le scan de l'enseignant clôture la session et empêche toute modification frauduleuse ultérieure des données de la séance.
- **US_05 : Envoi du Rapport de Présence par le Délégué**
- *En tant que Délégué de classe,*
- *je veux générer et envoyer le rapport consolidé basé sur le double scan à l'administration,*
- *afin de clore administrativement le cours.*
- *Critères d'acceptation* : L'application affiche un résumé clair (Nombre de présents, retardataires, absents) et intègre un bouton d'envoi réseau direct vers le système central de l'administration.\

6. ARCHITECTURE TECHNIQUE ET CONCEPTION DES DONNÉES

6.1. Choix Technologiques & Paradigme de Stockage

L'écosystème applicatif de **KeycePass** repose sur deux environnements distincts, connectés de manière décentralisée via le réseau Wi-Fi local :

- **Environnement Client Mobile (APK Android)** : Hébergé sur les smartphones des étudiants et délégués. Le stockage s'effectue via un moteur relationnel local léger **SQLite**, manipulé au travers de la bibliothèque ORM officielle **Room Database** en Kotlin.
- **Environnement d'Administration Desktop (EXE PC)** : Logiciel de bureau s'exécutant sur le poste de l'administration, conçu avec **Compose Multiplatform (CMP)**. Il embarque sa propre base de données relationnelle centralisée et expose un serveur d'écoute léger via le framework asynchrone **Ktor** pour collecter les paquets de données au format JSON.

6.2. Schéma Logique des Données (Tables et Colonnes)

Table	Attribut (Colonne)	Type de Données	Spécification & Rôle
Etudiant	id_etudiant	INT	Clé Primaire (Auto-incrément)
	matricule	VARCHAR	Unique — Identifiant académique officiel
	nom	VARCHAR	

Table	Attribut (Colonne)	Type de Données	Spécification & Rôle
	nom	VARCHAR	Nom de famille de l'étudiant
	prenom	VARCHAR	Prénom de l'étudiant
	classe_id	VARCHAR	Clé Étrangère logique (ex: "B2_IT")
	device_uuid	VARCHAR	Jeton d'empreinte matérielle (<i>Device Binding</i>)
Seance	id_seance	INT	Clé Primaire
	nom_matiere	VARCHAR	Libellé du cours (ex: "Ingénierie Logicielle")
	classe_id	VARCHAR	Indexation de la classe cible
	date_jour	DATE	Date de la séance (AAAA-MM-JJ)
	heure_debut	TIME	

Table	Attribut (Colonne)	Type de Données	Spécification & Rôle
	heure_fin	TIME	Heure de début officielle (HH:MM:SS)
	statut_seance	VARCHAR	Heure de fin officielle (HH:MM:SS)
			États : PLANIFIE, EN_COURS, CLOTURE_ENSEIGNANT
Emargement	id_emargement	INT	Clé Primaire
	etudiant_id	INT	Clé Étrangère reliée à la table Etudiant
	seance_id	INT	Clé Étrangère reliée à la table Seance
	horodatage_scan_debut	DATETIME	
	horodatage_scan_fin	DATETIME	Instant précis du premier pointage d'entrée

Table	Attribut (Colonne)	Type de Données	Spécification & Rôle
	statut_final	VARCHAR	Instant précis du second pointage de sortie Résultat calculé : EN_ATTENTE, PRESENT, RETARD, ABSENT

7. STRUCTURE DU PROJET (ORGANISATION MVVM MULTI-MODULES)

Afin de maximiser la réutilisation de code entre l'APK mobile et l'EXE d'administration sans duplication, le projet adopte l'arborescence multi-modules standardisée suivante sur GitHub :

```

KeycePass/
├── shared/                                # MODULE COMMUN : LOGIQUE MÉTIER
PARTAGÉE
│   ├── src/commonMain/kotlin/com/ak/keycepass/shared/
│   │   ├── domain/                      # Noyau métier pur et indépendant
des plateformes
│   │   │   ├── model/                  # Data classes (Etudiant.kt,
Seance.kt, Emargement.kt)
│   │   │   └── utils/                  # Algorithme central de calcul des
statuts (règle des 15 min)
│   │   └── network/                    # Modèles de données (DTO) JSON
pour les échanges Wi-Fi
│   └── androidApp/                      # MODULE MOBILE CLIENT (Génération
de l'APK)
│       ├── src/main/kotlin/com/ak/keycepass/android/
│       │   ├── data/                  # Couche d'accès aux données
physiques du smartphone
│       │   │   ├── local/              # Gestion de Room Database
(Interfaces DAO, Room-Entities)
│       │   │   └── repository/          # Implémentations concrètes des
dépôts de données Android

```



```

|      |— ui/                                # Interface Homme-Machine (IHM) en
Jetpack Compose
|      |   |— screens/                      # Écrans mobiles (ScanScreen,
LoginScreen)
|      |   |   |— components/              # Éléments réutilisables (Boutons
de scan, cartes graphiques)
|      |   |   |— viewmodel/              # ViewModels Android (Gestion
d'état réactif de l'UI mobile)
|      |   |       |— theme/              # Déclinaison graphique Material
Design 3 (Color.kt, Theme.kt)
|      |       |— navigation/            # Routage des écrans via Jetpack
Navigation Core
|
|— desktopApp/                              # MODULE ADMINISTRATION DESKTOP
(Génération de l'EXE)
|   |— src/main/kotlin/com/ak/keycepass/desktop/
|   |   |— data/                          # Gestion du serveur local et de
l'agrégation globale
|   |   |   |— server/                    # Moteur d'écoute réseau Ktor
(Réception des flux JSON)
|   |   |   |— database/                  # Connecteur et requêtes de la
base centrale sur PC
|   |   |       |— ui/                    # Interface d'Administration
(Compose for Desktop)
|   |   |   |— screens/                  # Écrans de bureau
(DashboardAdmin, GestionEnrolement)
|   |   |   |— components/              # Tableaux complexes et listes de
présence pour PC
|   |   |       |— viewmodel/            # ViewModels Desktop (Traitement
des flux administratifs PC)
|   |       |— navigation/              # Gestion des menus de contrôle
latéraux (Navigation Desktop)
|
|— build.gradle.kts                        # Script d'orchestration et de
gestion des dépendances
|— .gitignore                             # Fichier d'exclusion des fichier
|— readme.md                             # fichier explicatif du projet
ainsi que fonctionnement

```